AN INTERNSHIP REPORT

ON

Intellisql: intelligent sql querying with llms using gemini pro

Team ID : LTVIP2026TMIDS62173
Team Size : 4
Team Leader : Tummidi Srimanth
Team member : Mandela Yamini
Team member : Malladi Umesh Chandra
Team member : Tejasri Nookala

COLLEGE NAME: SWARNANDHRA COLLEGE OF ENGINEERING AND TECHNOLOGY

1. INTRODUCTION

1.1 Project Overview:

This project introduces **IntelliSQL**, an intelligent database querying platform designed to allow users to retrieve information from databases using natural language instead of writing complex SQL queries. The application leverages the power of Google's Generative AI model **Gemini Pro** to understand user questions and automatically convert them into executable SQL queries.

The platform enables users to:

- Input database-related questions in natural language
- Automatically convert English questions into SQL queries
- Execute the generated SQL queries on a relational database
- Retrieve and display structured data results instantly
- Interact with databases without requiring SQL knowledge

IntelliSQL transforms simple user questions into structured database queries by intelligently interpreting user intent and generating accurate SQL statements.

The system provides intelligent database interaction including:

- Natural Language to SQL Query Conversion
- Automated Query Generation using Gemini Pro
- Data Retrieval from relational databases
- Simplified database interaction for non-technical users
- Intelligent interpretation of user queries

The platform bridges the gap between **human language and structured database querying**, enabling users to access and analyse data efficiently without needing technical expertise in SQL.

1.2 Purpose

The purpose of IntelliSQL is to simplify database interaction and make data retrieval accessible to both technical and non-technical users. Many individuals struggle to extract information from databases because it requires knowledge of SQL and database structures. IntelliSQL addresses this challenge by:

- Providing instant AI-generated SQL queries from natural language questions
- Allowing users to retrieve database information without writing SQL code
- Enabling faster and more intuitive interaction with structured data
- Supporting efficient data analysis and decision-making

The project aims to bridge the gap between human language and database systems by enabling users to communicate with databases in a simple and natural way using AI-powered query generation.

2. IDEATION PHASE

2.1 Problem Statement:

In today's data-driven world, databases are widely used to store and manage large volumes of information. However, retrieving specific data from databases typically requires knowledge of SQL (Structured Query Language). Many users, especially non-technical individuals, find it difficult to write accurate SQL queries or understand complex database structures. This creates a barrier for users who need quick access to data but lack technical expertise. As a result, they depend on database experts or spend extra time learning SQL, which reduces efficiency and accessibility.

Many individuals face challenges such as:

- Difficulty in writing correct SQL queries
- Limited understanding of database structures and tables
- Time-consuming process of learning and debugging SQL syntax
- Inability to quickly retrieve required data from large databases
- Lack of user-friendly interfaces for interacting with databases

Traditional database systems require manual query writing and technical knowledge, which restricts easy access to information. This project addresses these challenges by integrating Generative AI with database systems, enabling users to ask questions in natural language and automatically converting them into SQL queries for efficient data retrieval.

2.2 User Needs and Challenges

In the context of database systems, users often require quick and easy access to stored information without dealing with the complexity of SQL queries. Many users, especially non-technical individuals, find it difficult to understand database structures and write accurate queries. This creates challenges in retrieving the required data efficiently.

Users typically need a system that allows them to interact with databases in a simple and intuitive way. They expect faster data retrieval, minimal technical effort, and a user-friendly interface that reduces the need for SQL knowledge.

This project addresses these needs by enabling users to ask questions in natural language and automatically converting them into SQL queries using Gemini Pro. This approach simplifies database interaction and makes data access more accessible and efficient.

2.3Brainstorming:

- Use Gemini Pro AI to convert natural language questions into SQL queries
- Design structured prompts to generate accurate and valid SQL queries
- Allow users to enter database questions in natural language

- Connect the system to a relational database (SQLite/MySQL) for query execution
- Automatically execute generated SQL queries and display results
- Provide a simple and user-friendly interface for database interaction

3.Requirement Analysis

3.1 Customer Journey Map

User visits the application interface
User enters a database question in natural language
System processes the user input and prepares a structured prompt
Prompt is sent to the Gemini Pro AI model
Gemini Pro converts the natural language question into an SQL query
The generated SQL query is executed on the connected database
Query results are retrieved and displayed to the user in a structured format.

3.2 Solution Requirement

Functional Requirements

- Accept natural language questions related to database queries
- Convert user queries into SQL statements using Gemini Pro
- Execute generated SQL queries on the database
- Retrieve and display query results to the user
- Handle basic database operations such as filtering, aggregation, and selection
- Provide a simple interface for interacting with the database

Non-Functional Requirements

- Fast response time for AI query generation
- Secure API communication with the Gemini service
- Reliable database connectivity
- User-friendly and intuitive interface
- Accurate SQL query generation

3.3 Data Flow Diagram

User Input → Natural Language Processing → Prompt Engineering → Gemini Pro API → SQL Query Generation → Database Execution → Query Results → User Interface

3.4 Technology Stack

• Frontend: Streamlit
• LLM API: Google Gemini API
• Model Used: Gemini Pro
• Framework Support: LangChain (optional for prompt management)
• Programming Language: Python 3.9+
• Database: SQLite / MySQL

4. PROJECT DESIGN

4.1 Overall Project Flow

The IntelliSQL system follows an AI-driven workflow that converts natural language questions into SQL queries and retrieves relevant data from a database using Google's Generative AI model Gemini Pro.

The system enables users to interact with databases without requiring knowledge of SQL syntax. By combining natural language processing with database execution, the platform simplifies data retrieval and improves accessibility.

The complete workflow is divided into three major stages.

4.2 User Interaction Layer

Natural Language Query Input

- Users access the application interface.
- A text input field is provided where users can enter their questions related to the database.
- Users can type queries in simple natural language (e.g., *Show all students with marks greater than 80*).
- The interface is designed to be simple, responsive, and user-friendly.
- This allows both technical and non-technical users to interact with the database easily.

4.3 Backend Processing Layer

Step 1: Prompt Creation

- The application receives the user's natural language query.
- The backend formats the input into a structured prompt for the AI model.
- A predefined prompt template ensures consistent and accurate SQL query generation.
- The prompt instructs the AI model to convert the natural language query into an SQL statement.
- Prompt engineering helps improve the accuracy and reliability of the generated SQL queries.

Step 2: Model Interaction

- The structured prompt is sent to the Gemini Pro AI model through the API.
- The system authenticates using a secure Google API Key.
- The AI model processes the request and interprets the user's intent.

- Gemini Pro generates the corresponding SQL query based on the user's question.

Step 3: Query Execution

- The generated SQL query is sent to the connected database.
- The database executes the SQL query.
- The system retrieves the relevant results from the database.
- The backend processes the results and prepares them for display.

4.4 User Output Layer

Display Query Results

- The query results are displayed in the application interface.
- The output is presented in a structured and readable format.
- Users can clearly view the data retrieved from the database.
- The system ensures quick and accurate data retrieval.
- This improves usability and allows users to interact with databases efficiently without SQL knowledge.

5. PROJECT PLANNING & SCHEDULING

To ensure systematic development and successful implementation, the IntelliSQL project (Intelligent SQL Querying with LLMs Using Gemini Pro) was divided into five key phases. Each phase includes specific objectives and deliverables to ensure smooth integration of AI, database connectivity, and user interface development.

Phase 1 – Research & Requirement Gathering

Timeline: Week 1

Objectives:

- Identify target users (students, database beginners, analysts, and non-technical users).
- Study existing database query systems and natural language interfaces.
- Identify limitations of traditional SQL-based database querying.
- Define functional and non-functional requirements of the system.
- Select tools and technologies (Python, Streamlit, Gemini API, SQLite/MySQL).

Phase 2 – AI Integration & Prompt Engineering

Timeline: Week 2

Objectives:

- Generate and configure Gemini API Key using Google AI Studio.
- Install required libraries (google-genai, streamlit, dotenv).
- Design structured prompt templates for:
    - Natural language to SQL query generation
    - Database schema understanding
    - Accurate query generation
- Test AI responses to ensure correct SQL query generation.

Phase 3 – Backend Development

Timeline: Week 3

Objectives:

- Develop logic to accept natural language user queries.
- Implement prompt formatting module for structured AI prompts.
- Integrate Gemini API for generating SQL queries in real time.
- Connect the system with a relational database (SQLite/MySQL).
- Execute generated SQL queries and retrieve results.
- Ensure secure API key management using environment variables.

Phase 4 – Frontend Development using Streamlit

Timeline: Week 4

Objectives:

- Develop the main Streamlit web interface.
- Create an input field for user queries.
- Add a submit button to process database questions.
- Display generated SQL queries and database results.
- Improve user interface readability and responsiveness.

Phase 5 – Testing and Optimization

Timeline: Week 5

Objectives:

- Test the accuracy of SQL query generation.
- Validate database connectivity and query execution.
- Measure AI response time and system performance.
- Fix prompt engineering issues for better query generation.
- Optimize user interface and improve overall system usability.

6. FUNCTIONAL AND PERFORMANCE TESTING

To ensure reliability, accuracy, and efficiency of the IntelliSQL system (Intelligent SQL Querying with LLMs Using Gemini Pro), both functional and performance testing were conducted. These tests verify that the system correctly converts natural language queries into SQL statements and retrieves accurate data from the database.

6.1 Functional Testing

Functional testing was performed to ensure that the system works according to the specified requirements.

Input Validation

- Verified that users can enter database questions in natural language.
- Ensured proper handling of empty or invalid inputs.
- Displayed appropriate error messages for incorrect or unsupported queries.

AI Query Generation Validation

- Confirmed that Gemini Pro correctly converts natural language queries into SQL statements.
- Verified correct SQL generation for operations such as:
    - Data retrieval
    - Filtering conditions
    - Aggregation functions (SUM, COUNT, AVG)

- Tested queries related to database tables and fields to ensure accuracy.

Database Execution Testing

- Ensured the generated SQL queries execute successfully on the connected database.
- Verified correct retrieval of records from the database tables.
- Checked that results match the expected database output.

Output Display Testing

- Ensured query results are displayed clearly in the application interface.
- Verified proper formatting and readability of results.
- Confirmed that users can easily interpret the retrieved data.

6.2 Performance Testing

Performance testing was conducted to ensure smooth system performance under normal usage conditions.

AI Response Time

- Measured the response time of the Gemini API for generating SQL queries.
- Average response time remained within acceptable limits (2–4 seconds).

UI Responsiveness

- Verified smooth interaction in the application interface.
- Ensured that the interface remains responsive while processing AI requests.

Scalability Testing

- Tested the system with multiple natural language queries.
- Confirmed system stability during extended usage sessions.

API Reliability

- Verified secure communication with the Gemini API.
- Ensured proper error handling when API requests fail or return invalid responses.

7. RESULTS

The final implementation of IntelliSQL successfully achieved the project objectives by enabling intelligent database interaction using natural language queries. The system allows users to retrieve information from databases without requiring knowledge of SQL syntax by leveraging the capabilities of Gemini Pro AI.

Key Outcomes:

• **Natural Language Query Processing**: Users can enter questions in simple natural language instead of writing complex SQL queries. The system automatically interprets the request and converts it into a valid SQL query.

• **Automated SQL Query Generation**: The Gemini Pro model generates SQL queries based on user inputs, reducing the need for manual query writing and minimizing errors in database interaction.

• **Efficient Data Retrieval**: The generated SQL queries are executed on the database, and the relevant data is retrieved and displayed instantly to the user.

• **User-Friendly AI Experience**

The Streamlit-based interface ensures that:

- Both technical and non-technical users can interact with the system easily.
- Query results are presented in a clear and structured format.
- Users receive quick access to database information without SQL knowledge.

The application provides fast, intelligent, and efficient database querying through AI-driven automation.

8. ADVANTAGES & DISADVANTAGES

Advantages

- AI-powered natural language to SQL conversion
- Simplifies database interaction for non-technical users
- Reduces time required to write complex SQL queries
- Improves accessibility and usability of database systems
- Scalable and adaptable to different database environments

Disadvantages

- Accuracy of SQL queries depends on prompt design
- Requires internet connectivity for AI model interaction
- May generate incorrect queries if the database schema is unclear
- Requires proper validation before executing generated SQL queries

9. CONCLUSION

The "Intelligent SQL Querying with LLMs Using Gemini Pro" project demonstrates how Generative AI can simplify database interaction by enabling users to query databases using natural language. By integrating the Gemini Pro AI model with a user-friendly interface, the system allows users to retrieve data efficiently without needing advanced SQL knowledge.

This project highlights the potential of AI-powered systems to improve accessibility to data and enhance the way users interact with database technologies. It represents a step toward intelligent data access systems that combine natural language understanding with database management.

10. FUTURE SCOPE

- Integration with multiple database systems (MySQL, PostgreSQL, SQL Server)
- Advanced query optimization and validation mechanisms
- Support for complex queries involving joins and nested operations
- Development of a mobile application for easier access
- Multi-language support for natural language queries
- Integration with business analytics dashboards

11. APPENDIX

Source Code & Resources

• GitHub Repository: ***(Add your project GitHub link here if available)***

• Tools and Libraries Used:

- **Python**
- **Streamlit**

- **Google Gemini API**
- **SQLite / MySQL**
- **LangChain (optional for prompt management)**